Extracting Domain Specific Parts from General Purpose Programs

Matei Budiu

University of Illinois Urbana-Champaign,
Illinois, USA,
mbudiu2@illinois.edu

Bhavya Hirani

University of Illinois Urbana-Champaign,
Illinois, USA,
bhirani2@illinois.edu

Charith Mendis

University of Illinois Urbana-Champaign,
Illinois, USA,
charithm@illinois.edu

Optimizing the matmul: One kernel to rule them all!

```
FOR I = 1, N:
  FOR J = 1, N:
    FOR K = 1, N:
      C[I, J] += A[I, K] * B[K, J]
    ENDFOR
  ENDFOR
ENDFOR
```

Optimizing the matmul: One kernel to rule them all!

```
FOR I = 1, N:                  ----- Dim 0
  FOR J = 1, N:                ----- Dim 1
    FOR K = 1, N:              ----- Dim 2
      C[I, J] += A[I, K] * B[K, J]
    ENDFOR
  ENDFOR
ENDFOR
```

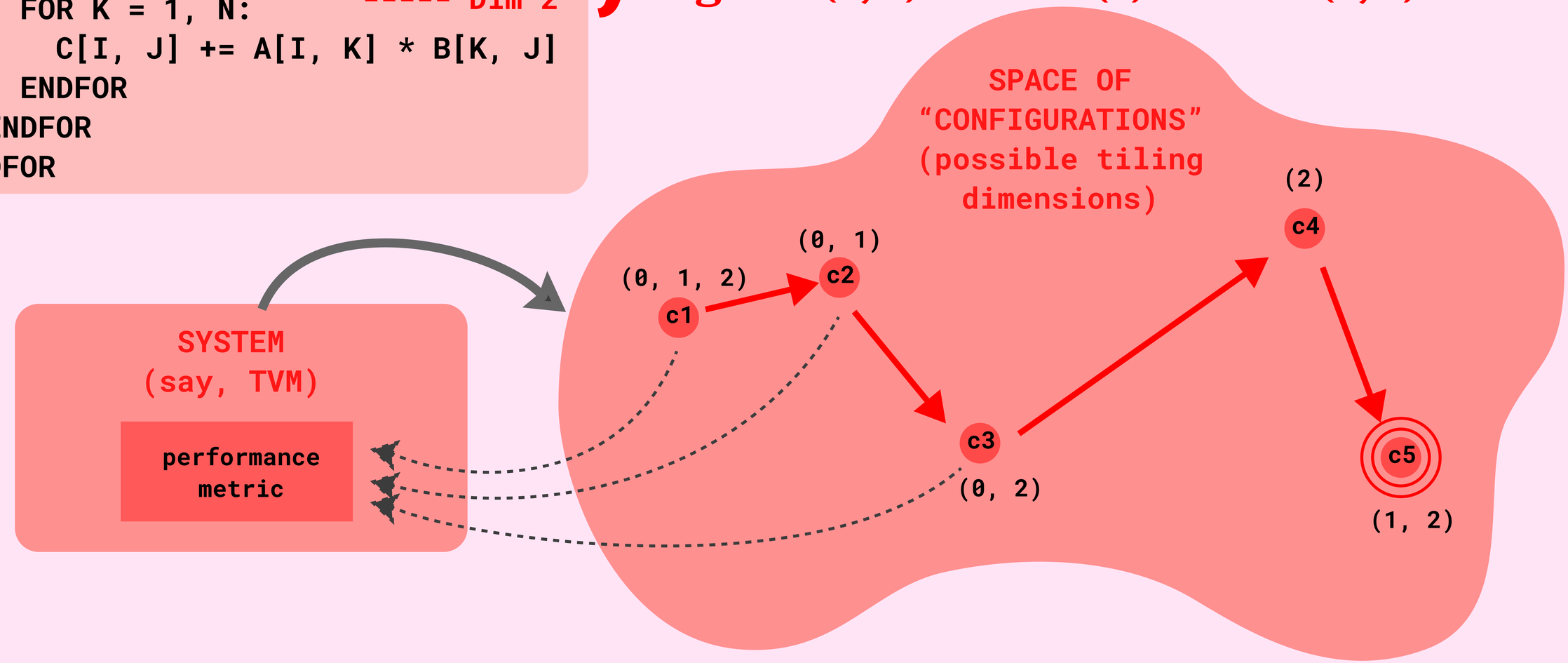
}

Different tiling combos possible
eg. tile (0, 1) OR tile (0) OR tile (1, 0)

Optimizing the matmul: One kernel to rule them all!

```
FOR I = 1, N:                  ----- Dim 0
  FOR J = 1, N:                ----- Dim 1
    FOR K = 1, N:              ----- Dim 2
      C[I, J] += A[I, K] * B[K, J]
    ENDFOR
  ENDFOR
ENDFOR
```

Different tiling combos possible
eg. tile (0, 1) OR tile (0) OR tile (1, 0)



And that’s not it!

Domain specific languages try to specialize for heavily-used operations in that domain so that they’re fast. Some optimizations include

- Reductions and scans
- Layout optimization
- Graph-level rewrites

And that's not it!

Domain specific languages try to specialize for heavily-used operations in that domain so that they're fast. Some optimizations include

- **Reductions and scans**
- **Layout optimization**
- **Graph-level rewrites**

However, they are hard to get used to and write code in, and are quite rigid.

```
// Are you ready? We're going to use all of the features abo
Func gradient_fast("gradient_fast");
gradient_fast(x, y) = x + y;

// We'll process 64x64 tiles in parallel.
Var x_outer, y_outer, x_inner, y_inner, tile_index;
gradient_fast
    .tile(x, y, x_outer, y_outer, x_inner, y_inner, 64, 64)
    .fuse(x_outer, y_outer, tile_index)
    .parallel(tile_index);
```

```
@tvm.script.ir_module
class MyModule:
    @T.prim_func
    def mm_relu(A: T.Buffer((128, 128), "float32"),
                B: T.Buffer((128, 128), "float32"),
                C: T.Buffer((128, 128), "float32")):
        Y = T.alloc_buffer((128, 128), dtype="float32")
        for i, j, k in T.grid(128, 128, 128):
            with T.sblock("Y"):
                vi = T.axis.spatial(128, i)
                vj = T.axis.spatial(128, j)
                vk = T.axis.reduce(128, k)
                with T.init():
                    Y[vi, vj] = T.float32(0)
                Y[vi, vj] = Y[vi, vj] + A[vi, vk] * B[vk, vj]
    for i, j in T.grid(128, 128):
        with T.sblock("C"):
            vi = T.axis.spatial(128, i)
            vj = T.axis.spatial(128, j)
            C[vi, vj] = T.max(Y[vi, vj], T.float32(0))
```

Our proposed solution

Extract domain specific parts from general purpose programs

Then use the domain specific compiler to optimize those parts!

Why is this a hard problem?
Why not replace these with hand-tuned libs?

```
FOR I = 1, N:
  FOR J = 1, N:
    FOR K = 1, N:
      C[I, J] += A[I, K] * B[K, J]
    ENDFOR
  ENDFOR
ENDFOR
```

Extract domain specific parts from general purpose programs

Then use the domain specific compiler to optimize those parts!

Why is this a hard problem?
Why not replace these with hand-tuned libs?

```
FOR I = 1, N:
  Arr[I] = Arr[I]+1
  FOR J = i, N:
    FOR K = 1, N:
      // other code
      C[I, J] += A[I, K] * B[K, J]
      T[i] = C[I, J]
    ENDFOR
    // more code
  ENDFOR
ENDFOR
```

The major challenge is extracting hidden (latent) representation, which programmers can't do manually

Extract domain specific parts from general purpose programs

Then use the domain specific compiler to optimize those parts!

TVM vs Polly SCoP

```
for t in T.serial(2048):
  for h in T.serial(16):
    for k in T.serial(2048):
      for rd in T.serial(512):
        with T.block("logits_n"):
          v_t = T.axis.spatial(2048, t)
          v_h = T.axis.spatial(16, h)
          v_k = T.axis.spatial(2048, k)
          v_rd = T.axis.reduce(512, rd)
          T.reads(
            q_nope[v_t:v_t+1, v_h:v_h+1, v_rd:v_rd+1],
            Kc_gather[v_t:v_t+1, v_k:v_k+1, v_rd:v_rd+1]
          )
          T.writes(
            logits_n[v_t:v_t+1, v_h:v_h+1, v_k:v_k+1]
          )
          logits_n[v_t, v_h, v_k] = (
            logits_n[v_t, v_h, v_k]
            + T.Cast("float32", q_nope[v_t, v_h, v_rd])
            * T.Cast("float32", Kc_gather[v_t, v_k, v_rd])
          )
```

```
Scop: {
  Statements:
    S_red[t, h, k, rd]:
      Domain:
        { S_red[t, h, k, rd] :
            0 <= t  < 2048 and
            0 <= h  < 16   and
            0 <= k  < 2048 and
            0 <= rd < 512 }
      ReadAccess:
        q_nope[t, h, rd]
        Kc_gather[t, k, rd]
        logits_n[t, h, k]
      WriteAccess (reduction):
        logits_n[t, h, k]
      Body:
        logits_n[t, h, k] =
            logits_n[t, h, k]
            + q_nope[t, h, rd]
            * Kc_gather[t, k, rd]
  Schedule:
     S_red[t, h, k, rd]
     -> (t, h, k, 1, rd)
}
```

TVM vs Polly SCoP

Loops

```
for t in T.serial(2048):
  for h in T.serial(16):
    for k in T.serial(2048):
      for rd in T.serial(512):
        with T.block("logits_n"):
          v_t = T.axis.spatial(2048, t)
          v_h = T.axis.spatial(16, h)
          v_k = T.axis.spatial(2048, k)
          v_rd = T.axis.reduce(512, rd)
          T.reads(
            q_nope[v_t:v_t+1, v_h:v_h+1, v_rd:v_rd+1],
            Kc_gather[v_t:v_t+1, v_k:v_k+1, v_rd:v_rd+1]
          )
          T.writes(
            logits_n[v_t:v_t+1, v_h:v_h+1, v_k:v_k+1]
          )
          logits_n[v_t, v_h, v_k] = (
            logits_n[v_t, v_h, v_k]
            + T.Cast("float32", q_nope[v_t, v_h, v_rd])
            * T.Cast("float32", Kc_gather[v_t, v_k, v_rd])
          )
```

```
Scop: {
  Statements:
    S_red[t, h, k, rd]:
      Domain:
        { S_red[t, h, k, rd] :
            0 <= t  < 2048 and
            0 <= h  < 16   and
            0 <= k  < 2048 and
            0 <= rd < 512 }
      ReadAccess:
        q_nope[t, h, rd]
        Kc_gather[t, k, rd]
        logits_n[t, h, k]
      WriteAccess (reduction):
        logits_n[t, h, k]
      Body:
        logits_n[t, h, k] =
            logits_n[t, h, k]
            + q_nope[t, h, rd]
            * Kc_gather[t, k, rd]
```

Schedule

(more info in AST)

```
Schedule:
   S_red[t, h, k, rd]
   -> (t, h, k, 1, rd)
}
```

TVM vs Polly SCoP

Loops

```
for t in T.serial(2048):
  for h in T.serial(16):
    for k in T.serial(2048):
      for rd in T.serial(512):
        with T.block("logits_n"):
```

Iter vars

```
v_t = T.axis.spatial(2048, t)
v_h = T.axis.spatial(16, h)
v_k = T.axis.spatial(2048, k)
v_rd = T.axis.reduce(512, rd)
T.reads(
  q_nope[v_t:v_t+1, v_h:v_h+1, v_rd:v_rd+1],
  Kc_gather[v_t:v_t+1, v_k:v_k+1, v_rd:v_rd+1]
)
T.writes(
  logits_n[v_t:v_t+1, v_h:v_h+1, v_k:v_k+1]
)
logits_n[v_t, v_h, v_k] = (
  logits_n[v_t, v_h, v_k]
  + T.Cast("float32", q_nope[v_t, v_h, v_rd])
  * T.Cast("float32", Kc_gather[v_t, v_k, v_rd])
)
```

Domain

```
Scop: {
  Statements:
    S_red[t, h, k, rd]:
      Domain:
        { S_red[t, h, k, rd] :
            0 <= t  < 2048 and
            0 <= h  < 16   and
            0 <= k  < 2048 and
            0 <= rd < 512 }
      ReadAccess:
        q_nope[t, h, rd]
        Kc_gather[t, k, rd]
        logits_n[t, h, k]
      WriteAccess (reduction):
        logits_n[t, h, k]
      Body:
        logits_n[t, h, k] =
            logits_n[t, h, k]
            + q_nope[t, h, rd]
            * Kc_gather[t, k, rd]
```

Schedule

(more info in AST)

```
      Schedule:
         S_red[t, h, k, rd]
         -> (t, h, k, 1, rd)
}
```

TVM vs Polly SCoP

Loops

```
for t in T.serial(2048):
  for h in T.serial(16):
    for k in T.serial(2048):
      for rd in T.serial(512):
        with T.block("logits_n"):
```

Iter vars

```
v_t = T.axis.spatial(2048, t)
v_h = T.axis.spatial(16, h)
v_k = T.axis.spatial(2048, k)
v_rd = T.axis.reduce(512, rd)
T.reads(
  q_nope[v_t:v_t+1, v_h:v_h+1, v_rd:v_rd+1],
  Kc_gather[v_t:v_t+1, v_k:v_k+1, v_rd:v_rd+1]
)
T.writes(
  logits_n[v_t:v_t+1, v_h:v_h+1, v_k:v_k+1]
)
```

Stmts

```
logits_n[v_t, v_h, v_k] = (
  logits_n[v_t, v_h, v_k]
  + T.Cast("float32", q_nope[v_t, v_h, v_rd])
  * T.Cast("float32", Kc_gather[v_t, v_k, v_rd])
)
```

```
Scop: {
  Statements:
    S_red[t, h, k, rd]:
```

Domain

```
Domain:
  { S_red[t, h, k, rd] :
      0 <= t  < 2048 and
      0 <= h  < 16   and
      0 <= k  < 2048 and
      0 <= rd < 512 }
ReadAccess:
  q_nope[t, h, rd]
  Kc_gather[t, k, rd]
  logits_n[t, h, k]
WriteAccess (reduction):
  logits_n[t, h, k]
```

Stmts

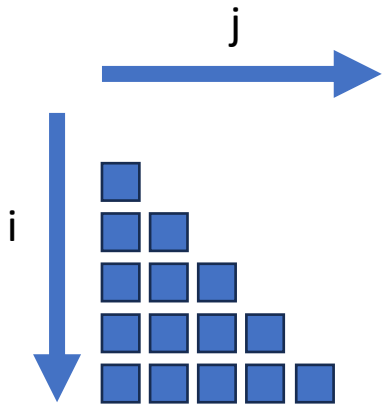
```
Body:
  logits_n[t, h, k] =
      logits_n[t, h, k]
      + q_nope[t, h, rd]
      * Kc_gather[t, k, rd]
```

Schedule

(more info in AST)

```
Schedule:
   S_red[t, h, k, rd]
   -> (t, h, k, 1, rd)
}
```

```
for (int i = 0; i < 6; i++) {
    for (int j = 0; j < i; j++) {
        a[i][j]++;
    }
}
```

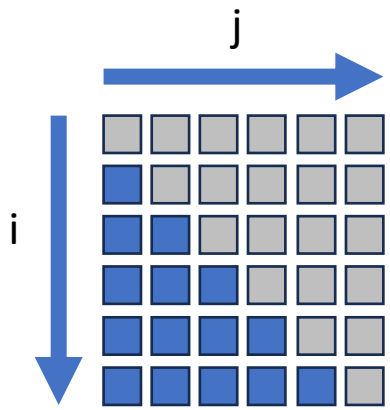


TVM (and the GPU) doesn't like this ☹

```
for (int i = 0; i < 6; i++) {
    for (int j = 0; j < 6; j++) {
        if (j < i)
            a[i][j]++;
    }
}
```

Use if statement masking

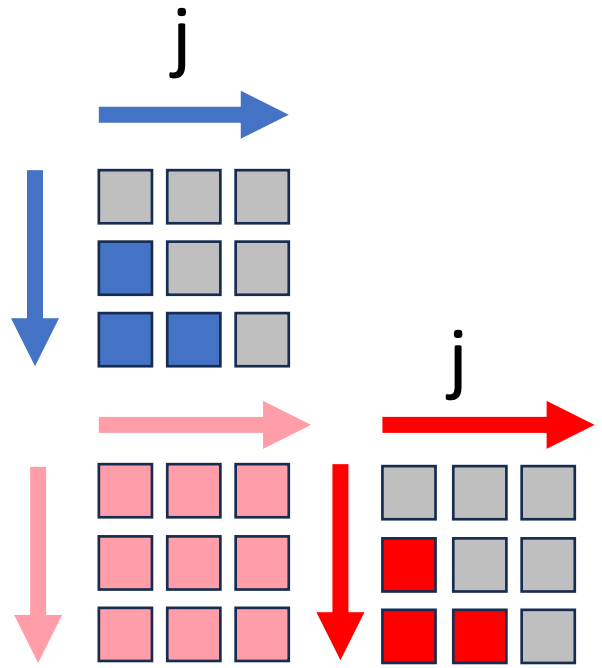
Find an enclosing rectangle bound



Non-rectangular bounds can be converted into rectangular ones by adding if statement masking

This results in less warp divergence on GPUs and is supported by TVM

```
for i = 0 to 3 {
    for j = 0 to i {
        if (j < i)
            a[i][j]++;
    }
}
for i = 3 to 6 {
    for j = 0 to 3 {
        a[i][j]++;
    }
}
```




```
for i = 3 to 6 {
    for j = 3 to i {
        if (j < i)
            a[i][j]++;
    }
}
```

Another strategy: create multiple rectangles (minimize masked iterations)

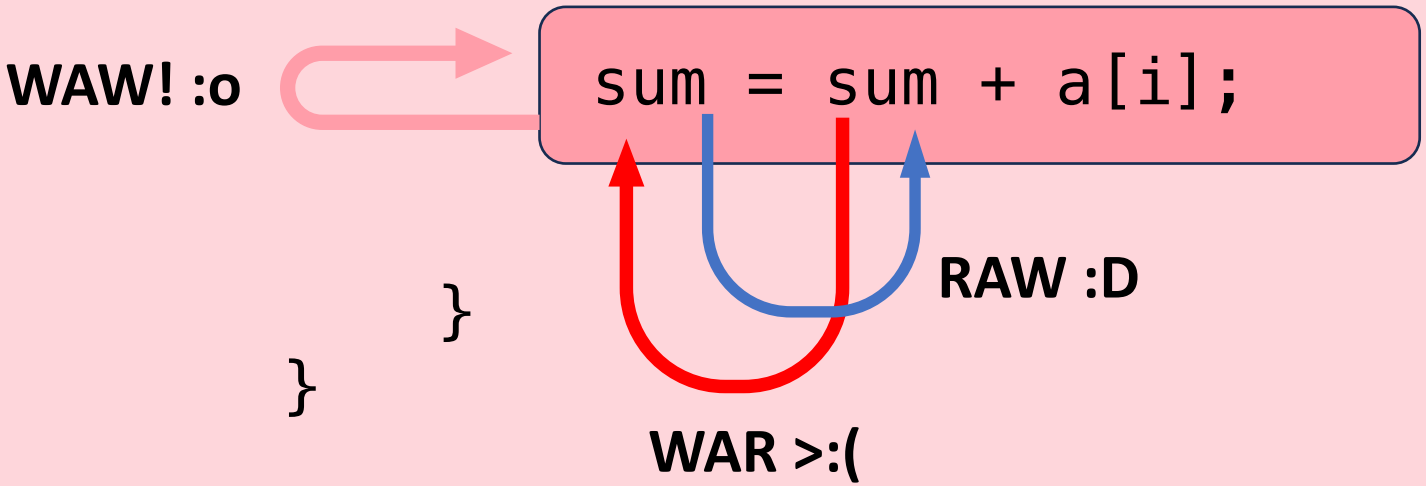
Downsides:

- Increases program size and auto-tuning search space
- Profitability is questionable
- Hard to choose a good set of bounding boxes

Let us have a look at reductions

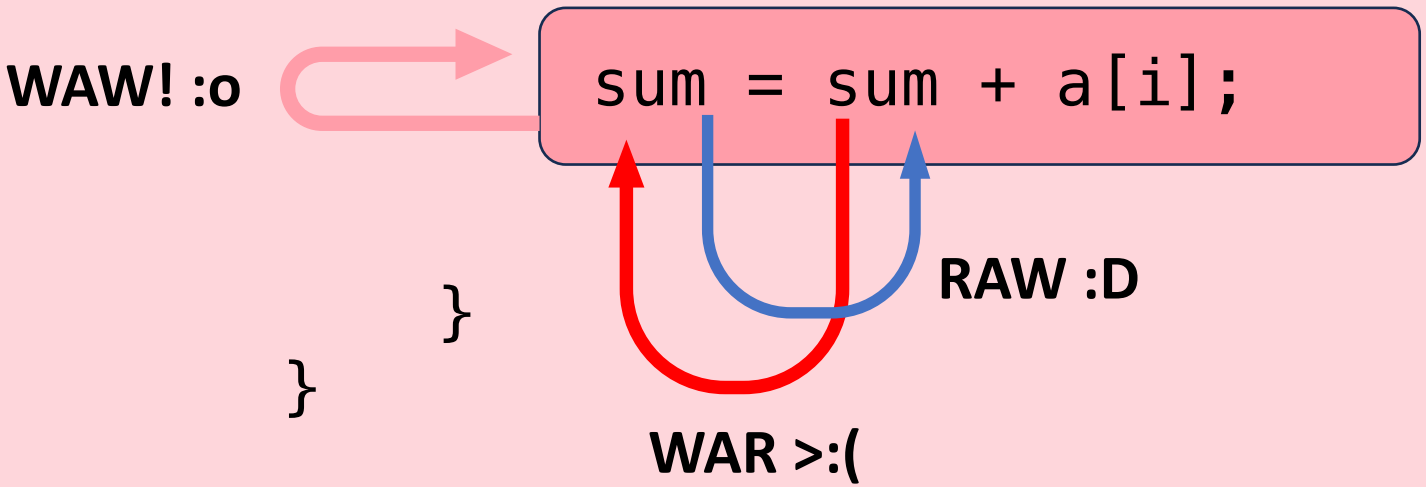
(Since apparently we didn't see enough dependencies yesterday)

```
void reduction(int *a, int n) {
    for (int i = 1; i < n; i++) {
        sum = sum + a[i];
    }
}
```



Polly gives us reduction detection for “free”! *Waw!*

```
void reduction(int *a, int n) {
    for (int i = 1; i < n; i++) {
        sum = sum + a[i];
    }
}
```



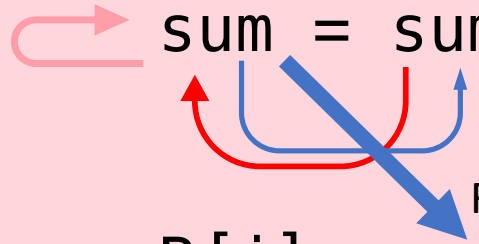
However…what if we add another statement?

```
void reduction(int *a, int n) {
    for (int i = 1; i < n; i++) {

        sum = sum + a[i];

        B[i] = sum;

    }
}
```



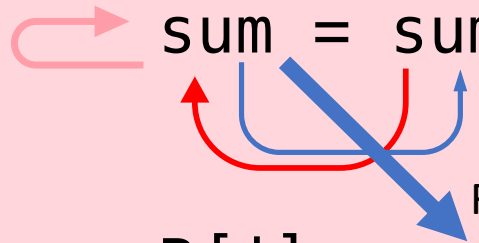
However…what if we add another statement?

```
void reduction(int *a, int n) {
    for (int i = 1; i < n; i++) {

        sum = sum + a[i];

        B[i] = sum;

    }
}
```



In this case, the leaking partial sums make this a scan.

TVM needs to know about scans, but Polly does not recognize them.

Takeaways

- Domain specific languages can be used to accelerate some types of kernels
- However, it can be hard to write code in them
- Hence, we extract and translate general purpose program parts into DSLs

“LaTeaVM”